Range-Vectorized Discounted Regret Minimization for No-Limit Hold'em: Complexity, Precision and Parallelism

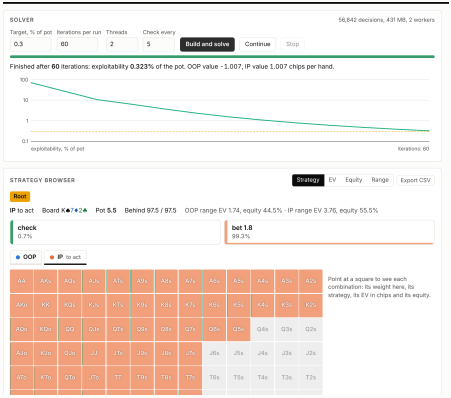
Numerical Analysis Research Club

Background and scope

Programs based on counterfactual regret minimization (CFR) defeated professional players at heads-up no-limit hold'em in 2017 and at six-player hold'em in 2019.

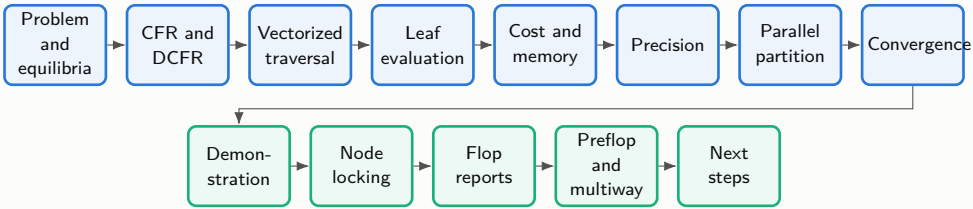
Commercial solvers compute equilibria of user-specified subgames: a board, two ranges and a set of bet sizes.

We built such a solver with one algorithm, discounted CFR (DCFR), for the flop, turn and river, with node locking, aggregate reports and a preflop solver for two to six players. This talk analyses its numerics.



The postflop solver of this work, running in a browser on a flop spot.

Outline



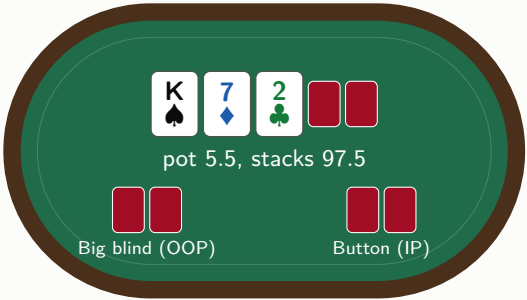
The first row is the solver core and its analysis. The second row shows the core at work and the extensions built on it.

The postflop subgame

Each player holds two private cards. The board has three, four or five public cards, and the remaining cards are dealt by chance between betting rounds.

A subgame is given by the board, the pot, the effective stack, a **range** for each player and a **bet tree** of allowed sizes. A range is a weight vector over the at most 1081 hands that are consistent with the board.

The unknown is an action distribution for every hand at every decision node.



street	decision nodes	hands
river	8	449 / 428
turn	2,216	474 / 448
flop	56,842	488 / 460

Zero-sum games, equilibrium and exploitability

Behavioural strategies of the two players determine realization plans $x \in X$ and $y \in Y$, polytopes defined by flow constraints, and the expected payoff is bilinear.

Minimax theorem (von Neumann 1928)

$$\max_{x \in X} \min_{y \in Y} x^\top A y \;=\; \min_{y \in Y} \max_{x \in X} x^\top A y \;=:\; v^*.$$

A pair attaining both sides is a Nash equilibrium. Solution quality is measured by the **exploitability**

$$\mathrm{expl}(x, y) \;=\; \tfrac{1}{2}\Big(\max_{x'} x'^\top A y \;-\; \min_{y'} x^\top A y' \Big) \;\geq 0,$$

reported as a percentage of the pot. With rake the game is not zero-sum, and we report the mean gain of the two best responses.

Regret matching and counterfactual regret minimization

At every information set I, CFR keeps a cumulative regret per action and plays in proportion to its positive part:

$$R_T(I,a) = \sum_{t=1}^{T} \big(v_t(I,a) - v_t(I)\big), \qquad \sigma_{T+1}(I,a) = \frac{[R_T(I,a)]^+}{\sum_b [R_T(I,b)]^+}.$$

The values v_t are **counterfactual**: they are weighted by the probability that chance and the opponent reach I.

Theorem (Zinkevich, Johanson, Bowling and Piccione 2007)

The overall regret is bounded by the sum of the counterfactual regrets, so the average strategies satisfy

$$\mathrm{expl}(\bar{x}_T, \bar{y}_T) \;\le\; \frac{\Delta\,|\mathcal{I}|\sqrt{|A|}}{\sqrt{T}}.$$

One iteration evaluates Ay and $A^\top x$ by a traversal of the tree, without forming A.

Discounted CFR

DCFR discounts the cumulative quantities at iteration t before adding the new ones:

$$R \leftarrow \begin{cases} R\,\dfrac{t^{\alpha}}{t^{\alpha}+1} & R > 0 \\ R\,\dfrac{t^{\beta}}{t^{\beta}+1} & R \le 0 \end{cases} + \big(v(a) - v\big), \qquad S \leftarrow S\Big(\frac{t}{t+1}\Big)^{\gamma} + \pi_{\text{own}}\,\sigma_t.$$

We use $(\alpha, \beta, \gamma) = (1.5, 0, 2)$ and update the players alternately. CFR is the limit $\alpha = \beta = \infty$, $\gamma = 0$, and CFR$^+$ is $\alpha = \infty$, $\beta = -\infty$, $\gamma = 1$.

Theorem (Brown and Sandholm 2019)

With $(\alpha, \beta, \gamma) = (1.5, 0, 2)$ the weighted average strategies form a $6\Delta|\mathcal{I}|\big(\sqrt{|A|} + 1/\sqrt{T}\big)/\sqrt{T}$ equilibrium after T iterations.

The worst-case rate is unchanged; the gain is empirical and is measured later in the talk.

Range-vectorized traversal of the public tree

The **public tree** has one node per betting sequence and dealt card. At a public node the information sets of the acting player are indexed by its hands, so every quantity is a vector of length n_p.

A traversal for player p carries the opponent's reach $r_{-p} \in \mathbb{R}^{n_{-p}}$ down and returns the values $v_p \in \mathbb{R}^{n_p}$ up. A decision node with $|A|$ actions costs $O(n|A|)$ arithmetic on contiguous memory.

Regrets and strategy sums are stored action-major, so the inner loops run over hands.

One traversal for player p

Opponent node. For each action a, recurse with $r_{-p} \odot \sigma(a)$ and add the child values.

Own node. Recurse on every action, set $v = \sum_a \sigma(a) \odot v_a$ and update R and S.

Chance node. Average the children over the dealt cards.

Leaf. Return the payoff against r_{-p}.

Chance nodes

Once both hands are fixed, $K = 48 - |\mathrm{board}|$ cards remain, and each of them is equally likely.

Lemma

For hands h_1, h_2 and a card c not in $h_1 \cup h_2 \cup \mathrm{board}$, $\Pr(c \mid h_1, h_2, \mathrm{board}) = 1/K$.

Hence the chance node returns

$$v_p(h) \;=\; \frac{1}{K} \sum_{c \notin \mathrm{board}} \mathbf{1}[c \notin h]\; v_p^{(c)}(h),$$

where the child for card c is evaluated with the opponent's hands containing c set to zero reach. The weight is the same constant for every hand pair, so chance needs no normalization by range.

Fold leaves in linear time

At a fold leaf the value of hand h is the payoff times the opponent weight that shares no card with h:

$$m(h) \;=\; \sum_{j\,:\,j\cap h=\emptyset} r_j \;=\; T - C(h_1) - C(h_2) + r_{\bar{h}}, \qquad T = \sum_j r_j, \quad C(c) = \sum_{j\,\ni\,c} r_j.$$

The last term restores the opponent hand equal to h, which was subtracted twice.

The 52 card sums cost $O(n)$, and so does the evaluation for all h. The direct sum costs $O(n^2)$.

In matrix form the compatibility matrix is

$$C = \mathbf{1}\mathbf{1}^\top - \sum_{c=1}^{52} e_c f_c^\top + P,$$

where e_c and f_c indicate the hands containing card c and P matches identical hands: rank at most 53 plus a partial permutation.

Showdown leaves in linear time

Sort both lists by hand strength once per runout. One ascending sweep gives, for each own hand, the compatible opponent weight that is strictly weaker, $L(h)$, and weaker or equal, $L^{=}(h)$, with the same inclusion–exclusion inside the sweep. Then

$$v(h) = \ell\, m(h) + (w-\ell)\, L(h) + (\tau-\ell)\big(L^{=}(h) - L(h)\big),$$

where w, ℓ and τ are the payoffs for a win, a loss and a tie.

The sort is cached per runout, so each evaluation is $O(n)$.

n	fold	showdown	direct
50	0.4	1.2	12
100	0.6	1.8	53
200	1.1	3.1	281
400	2.6	5.6	1326
700	3.6	9.6	4056
1081	7.0	14.4	4880

Time per leaf evaluation in microseconds for n hands.

The direct method sums over all compatible pairs; the largest relative difference is 7×10^{-13}.

Cost per iteration

One iteration is one traversal per player, with cost

$$W = \sum_{\text{decisions}} n_p |A| + \sum_{\text{leaves}} O(n) + \sum_{\text{chance}} O(Kn).$$

tree	decisions	$\sum n_p \lvert A \rvert$	MB	ms per iteration	ns per entry
river, 1 size	4	3,508	0.03	0.9	249
river, 2 sizes, raises	8	8,770	0.07	0.4	43
turn, 1 size	580	5.3×10^5	4	23	42
turn, raises and all-in	2,216	2.4×10^6	20	97	40
flop, checked down later	8	9,480	0.08	269	28394
flop, 1 size per street	56,842	5.4×10^7	431	1759	33
flop, standard	85,263	8.1×10^7	647	1743	22

Measured on one core. On trees with betting on every street the time per regret entry is roughly constant, about 33 ns including the leaves. With the later streets checked down the leaves dominate.

Memory

Each decision node stores regrets and strategy sums for every hand and action. In single precision this is

$$\text{memory} \;=\; 8 \sum_{\text{decisions}} n_p \,|A| \text{ bytes.}$$

Storage is a few flat blocks of 2^{24} entries, and each node holds two views into them. This cut the allocation time of a 2.9 GB tree from 32 s to under a second.

The interface estimates the memory before building, from the top of the tree and one card's subtree.

flop	turn	river	MB
— / 33	75 / 75	75 / 75	431
x/r 50 / 33	75 / 75	75 / 75	647
x/r 50 / 33	– / 75	75 / 75, all-in	755
x/r 50 / 33	66 / 66, r 50	75 / 75, all-in	1,457
x/r 50 / 33, 75	– / 75, r 50	75, all-in	1,681
x/r 60 / 33	75, r 60	75, r 60, all-in	3,614

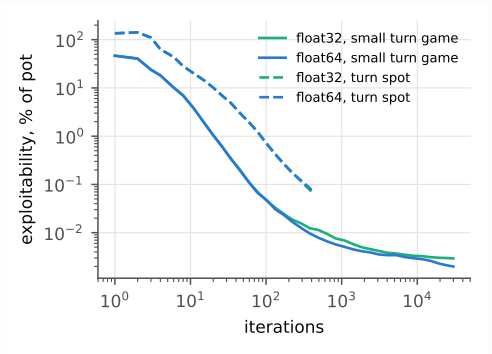
Flop K72, the big blind against the button. Sizes are percentages of the pot for out of position / in position.

Floating-point precision

Regrets and strategy sums are stored in single precision; all arithmetic inside a traversal is double precision.

Each update has relative rounding error $u = 2^{-24} \approx 6 \times 10^{-8}$. Under DCFR the strategy sum grows like $t/3$ and the new term has relative size about $3/t$, so it is resolved while $3/t \gg u$, that is for $t \ll 10^7$. Regrets grow like $\sqrt{t}$ and are resolved for longer.

The two storage precisions give exploitabilities within 10% of each other down to 10^{-2}% of the pot, ten to fifty times below the tolerances used in practice. Only in long runs below that level does double precision gain.



Exploitability against iterations with single and double precision storage.

Parallel partition by the first chance card

Below the first chance node the subtrees for different cards are independent. Cards are assigned to P workers and a coordinator keeps the tree above. One traversal is three steps:

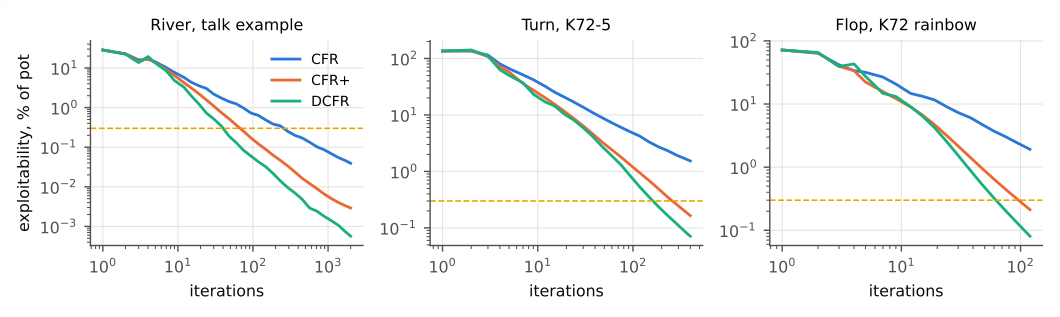
1. the coordinator records both reach vectors at each first-level chance node;
2. each worker returns, per chance node, the sum of its cards' child values;
3. the coordinator divides by K and finishes the traversal, updating its nodes.

The arithmetic is that of the sequential traversal up to the order of summation.

spot	sequential	$P = 1$	$P = 2$	speedup	same iterates
turn	65	84	68	0.96	no
flop	2011	2518	1311	1.53	no

Time per iteration in milliseconds on the 2-core test machine. The communication per traversal is two vectors per first-level chance node and worker.

Convergence of CFR, CFR$^+$ and DCFR

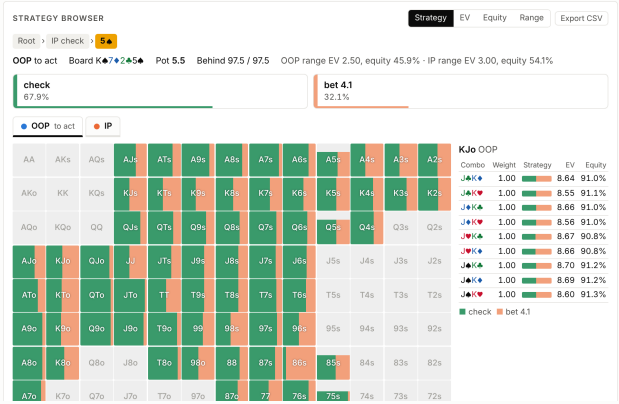


Exploitability in percent of the pot against iterations, on the river of the earlier talk, a turn spot and the K72 flop. Fitted slopes over the second half of each run: CFR $-0.98, -0.84, -0.98$; CFR$^+$ $-1.38, -1.42, -1.69$; DCFR $-1.53, -1.75, -2.15$ for river, turn and flop.

Validation

test	reference	result
River, talk example	the earlier independent solver	same strategies, exploitability 1.38×10^{-4} after 5000 iterations
Turn with chance nodes	exact sequence-form LP in Python	game value 24.92419856 against 24.92419863
Partitioned solving	sequential solver	identical iterates on river, turn and flop
Node locking	sequential solver with the same lock	identical values
Heads-up push or fold	linear program on the same tables	values agree to five digits
Leaf evaluation	direct $O(n^2)$ sums	agreement to 10^{-12}

Demonstration



Strategy browser after a flop solve: the big blind's turn strategy after the flop checks through.

The site has five pages: postflop, preflop and multiway, flop reports, the river table of the earlier talk and a methods page. In the demonstration we

1. solve a flop and follow the exploitability,
2. browse strategies, EV and equity by turn card,
3. lock a node and re-solve,
4. solve a push-or-fold game and a six-player game.

Node locking

A locked node keeps a fixed strategy $\hat{\sigma}$: its regrets are not updated, the traversals use $\hat{\sigma}$, and best responses do not optimize over it. DCFR then solves the **restricted game**

$$\min_{y \in Y} \max_{x \in X,\ x|_{\text{locked}} = \hat{\sigma}} x^\top A y,$$

and the exploitability is measured in the restricted game.

Example on the river: after a check the button is locked to bet 33 with every hand.

	equilibrium	against the lock
button bets after a check	0.0%	100%
big blind folds to that bet	49.2%	29.4%
big blind value per hand	-16.23	-6.62

Aggregate flop reports

The symmetric group S_4 acts on the suits. Its orbits on the $\binom{52}{3} = 22{,}100$ flops are the **1755** strategically different flops, and each orbit has size 4, 12 or 24.

A report solves one spot on a subset and averages with weights equal to the orbit sizes it represents. Subsets are stratified by texture and ordered by rank, so each stratum is represented in proportion to its weight.

flops	big blind value	big blind bets
25	-0.450	20.4%

Weighted averages from stratified subsets, on the flop tree with checked-down turn and river. The big blind's value is in chips per hand with a pot of 5.5.

Preflop: card removal between hand classes

Preflop strategies are per class, 169 of them. A hand that sees a flop is settled by all-in equity, so postflop play is not modelled. Card removal between classes i and j is the exact fraction of compatible pairs

$$M_{ij} = \frac{\#\{\text{compatible pairs of combinations}\}}{|i|\,|j|}, \qquad M = \mathbf{1}\mathbf{1}^{\top} - D,$$

where D is nonzero only for classes that share a rank (27% of the entries). With two players this is exact.

stack	LP value	DCFR value	push	call
5 bb	0.05616	0.05616	71.5%	62.0%
10 bb	-0.04538	-0.04538	58.2%	37.3%
15 bb	-0.12721	-0.12721	45.2%	28.5%
20 bb	-0.18316	-0.18316	40.2%	21.8%

Heads-up push or fold. DCFR agrees with a linear program on the same tables, and the frequencies match published equilibrium charts.

Multiway: three-way equity and convergence

A three-way showdown needs $\sum_{j,k} a_j b_k F_i(j,k)$ for every i, where $F_i(j,k)$ is the equity of i against j and k: 169^3 operations per leaf. A symmetric eigendecomposition truncated to rank R,

$$F_i \approx \sum_{r=1}^{R} \lambda_{ir}\, u_{ir} u_{ir}^{\top},$$

reduces this to $2 \cdot 169 \cdot R$ per class.

R	slice error	contraction error
1	0.19	0.016
4	0.14	6.3×10^{-3}
8	0.10	4.8×10^{-3}
16	0.059	2.7×10^{-3}
32	0.047	2.5×10^{-3}

With three or more players DCFR has no convergence guarantee. We measure NashConv, the sum of the players' best-response gains.

game	nodes	ms/it.	iter.	NashConv
3 players, push/fold	6	4	843	4.6×10^{-7}
6 players, push/fold	55	72	843	1.4×10^{-6}
6 players, 20 bb	1,197	66	843	0.061
6 players, 100 bb	6,842	303	219	0.049

Limitations

- Running time on the flop is seconds per iteration in a browser and grows with every added bet size. Commercial solvers are native code with vector instructions and suit isomorphism.
- The worst-case rate of DCFR is $O(T^{-1/2})$. The fast observed rates are not explained by theory, and the last digits are expensive.
- Preflop play treats postflop as an all-in showdown, so it overvalues hands that realize equity poorly.
- Multiway solutions have no equilibrium guarantee and use an independence approximation for card removal between opponents.
- The equity tables are Monte Carlo estimates with errors near 0.2% for two players and 1.5% per entry for three.

Next steps

- **High-precision finish.** Restrict to the DCFR support and solve the equilibrium conditions with a proximal semismooth Newton method, as in our earlier work on river subgames, which reached 10^{-13} with a best-response certificate.
- **Suit isomorphism.** Solve one representative of each class of isomorphic turn and river cards, which saves up to a factor of four on monotone flops.
- **Compiled kernels.** WebAssembly with SIMD for the vector loops, and GPU kernels for the leaf sweeps.
- **Pruning.** Skip subtrees with negligible reach or persistently negative regret.
- **Preflop with realization.** Replace the all-in showdown by values from postflop solves.
- **Theory.** Explain the observed rates of DCFR on poker trees.

Conclusions

- One method, discounted CFR, solves flop, turn and river subgames, locked games and preflop games.
- Range vectorization makes each iteration a sequence of vector operations, and inclusion–exclusion with sorted sweeps makes every leaf linear in the number of hands.
- The cost per iteration is proportional to the number of regret entries, single precision storage is sufficient at practical tolerances, and the partition by chance card is exact.
- DCFR reaches practical accuracy in a few hundred iterations on every street.

Site, source and experiments: `github.com/dwangnumbers/Poker-Solver`

References

- D. Koller, N. Megiddo, B. von Stengel. Efficient computation of equilibria for extensive two-person games. *Games Econ. Behav.* 14 (1996).
- S. Hart, A. Mas-Colell. A simple adaptive procedure leading to correlated equilibrium. *Econometrica* 68 (2000).
- M. Zinkevich, M. Johanson, M. Bowling, C. Piccione. Regret minimization in games with incomplete information. *NeurIPS* 2007.
- M. Johanson, K. Waugh, M. Bowling, M. Zinkevich. Accelerating best response calculation in large extensive games. *IJCAI* 2011.
- O. Tammelin. Solving large imperfect information games using CFR$^+$. arXiv:1407.5042 (2014).
- N. Brown, T. Sandholm. Solving imperfect-information games via discounted regret minimization. *AAAI* 2019.
- N. Brown, T. Sandholm. Superhuman AI for multiplayer poker. *Science* 365 (2019).

Appendix: the showdown sweep

Input: own list sorted by strength s, opponent list sorted by strength, opponent reach r.

1. Set $\mathrm{cum} = 0$ and $C(c) = 0$ for all cards.
2. For each block of own hands with equal strength s:
   2.1 add every opponent hand j with $s_j < s$ to cum and to C of its two cards;
   2.2 for each own hand h of the block set $L(h) = \mathrm{cum} - C(h_1) - C(h_2)$;
   2.3 add every opponent hand with $s_j = s$, and set $L^{=}(h) = \mathrm{cum} - C(h_1) - C(h_2) + r_{\bar{h}}$.
3. After the sweep, $m(h)$ is the fold formula with the final sums.

Each opponent hand is added once and each own hand is read twice, so the cost is $O(n_1 + n_2)$.

Appendix: counting flops with Burnside's lemma

The number of orbits of S_4 on flops is the average number of flops fixed by a suit permutation g:

$$\#\text{orbits} = \frac{1}{24}\sum_{g \in S_4} |\mathrm{Fix}(g)| = \frac{22100 + 6 \cdot 2938 + 8 \cdot 299}{24} = 1755.$$

cycle type	count	flops fixed
identity	1	all 22,100
transposition $(a\,b)$	6	$\binom{26}{3}$ in the other two suits, plus a pair of equal rank in a and b with one more card: $2600 + 338$
3-cycle	8	$\binom{13}{3}$ in the fixed suit, plus one rank in the three cycled suits: $286 + 13$
double transposition, 4-cycle	3, 6	none

The code maps each flop to its smallest image under the 24 permutations and finds the same 1755 classes.

Appendix: the earlier high-precision results

The previous talk solved river subgames to machine precision with a proximal semismooth Newton method started from DCFR, with interior-point crossover and support restriction.

game	DCFR	DCFR then Newton
Leduc hold'em	1.0×10^{-5} in 14 s	5.5×10^{-14} in 0.33 s
small river	5.4×10^{-6} in 9 s	3.9×10^{-14} in 1.9 s
river of the talk	1.3×10^{-4} in 16 s	3.2×10^{-5} in 528 s

Exploitability in chips with a pot of 100. On the largest river the Newton phase was slower than DCFR, which is why the method is listed as a next step and not as part of the solver.